

SGLang 前缀缓存实验与请求流程

第二次挑战 · 姜佳岩 · 2026-10-03 更新

本报告记录 2026 年 10 月 1 日完成的 run-1。任务一的数据来自本地服务的 64 条测量请求；任务二按官方 v0.5.14 源码梳理。作业感受第（2）、（3）问采用本人提供的文字，AI 辅助情况在独立说明中记录。

1 实验目的与环境

比较共享前缀和分散前缀在相同输入长度、输出长度与并发数下的性能，检查 Radix Cache 是否减少 Prefill 工作量，并把观测结果对应到请求处理流程。

项目	本次配置
运行环境	Windows 中的 WSL / Ubuntu-24.04
GPU	NVIDIA GeForce RTX 4060 Laptop GPU, 8 GB 显存
Python / PyTorch	3.12.3 / 2.11.0
SGLang / Ray	0.5.14 / 2.56.0
CUDA runtime / nvcc	13.0 / 13.0.88
模型与服务	Qwen/Qwen3-0.6B; 127.0.0.1:30000
服务设置	context-length=4096; mem-fraction-static=0.6; stream-interval=1; Radix Cache 开启

2 两组负载与测量步骤

每组 32 条请求，每条输入 2112 token，输出固定为 16 token，客户端最大并发为 8。共享组由 2048 token 公共前缀和 64 token 独立后缀组成；分散组的首 token 各不相同，其余部分与共享组逐条配对一致。后缀首 token 也各不相同，避免把后缀的一部分计入共享前缀。

通过原生 /generate 发送 input_ids，使用 SSE 流式响应。固定 temperature=0、max_new_tokens=16、ignore_eos=true、sampling_seed=2026。负载构造也使用种子 2026。客户端用标准库线程池；Ray 按题目版本安装，但本脚本没有使用 Ray 任务 API。

正式测量前先发送短请求预热服务。每组开始前等待已有请求结束，POST /flush_cache，确认 HTTP 200 和清空成功文本。共享组再发送一条 2112 token 的独立预热请求，其中前 2048 token 为公共前缀；分散组不做前缀预热。两组按 index 0 至 31 的顺序发送。探测请求、服务预热和前缀预热均不进入主表。

任务一：测量结果

表 1 两组对照结果（各 32 条测量请求，run-1）

指标	共享前缀	分散前缀
成功请求 / 成功率	32 / 100%	32 / 100%
测量区间 (s)	0.661817	4.028142
吞吐量 (请求/s)	48.3517	7.9441
输出吞吐量 (token/s)	773.6276	127.1057
输入 token 总数	67,584	67,584
输出 token 总数	512	512
缓存命中 token 总数	65,536	0
缓存命中率	96.9697%	0%
Prefill token 总数	2,048	67,584
TTFT p50 (ms)	43.313	533.409
TTFT p95 (ms)	55.494	793.492
TPOT p50 (ms)	7.646	31.016
TPOT p95 (ms)	8.608	53.203
端到端延迟 p50 (ms)	161.406	1009.126
端到端延迟 p95 (ms)	169.272	1020.663

指标口径

缓存命中率 = $\Sigma \text{cached_tokens} / \Sigma \text{prompt_tokens}$ ；Prefill token 总数 = $\Sigma(\text{prompt_tokens} - \text{cached_tokens})$ 。这是题目规定的 token 计数口径，未直接测量 GPU 算子的执行次数。吞吐量的时间区间为首条测量请求开始发送至最后一条流式响应结束，不包含预热。

TTFT 从发送请求前计时至首个输出 token 事件到达客户端；TPOT = (最后一个输出 token 到达时间 - 首 token 到达时间) / 15；端到端延迟计至流结束。p50、p95 对 32 个成功请求排序后按 $(n - 1) \times q$ 线性插值。表中时间从秒换算为毫秒。

原始数据检查：64 条均为 HTTP 200，输入均为 2112 token、输出均为 16 token；首个流式事件均包含 1 个输出 token；两组实际峰值并发均为 8。未发现请求回撤。工作负载 SHA256 与 manifest 一致，逐请求记录、原始 SSE 事件和汇总统计相互吻合。

表格对应 results/target1/{shared_prefix,dispersed_prefix}/run-1/summary.json；逐请求数据为 requests.jsonl，原始事件在 streams/。额外核对结果在 results/target1/verification.json。

任务一：原理与分析

3 相同前缀为什么可以复用 K/V

因果自注意力可写为下式，其中 M 是因果掩码：

$$\begin{aligned} \text{Attention}(Q, K, V) &= \text{softmax}(QK^T / \sqrt{d_k} + M)V \\ K_i &= h_i W_K, \quad V_i = h_i W_V \\ M_{ij} &= 0 \quad (j \leq i), \quad M_{ij} = -\infty \quad (j > i) \end{aligned}$$

因果掩码让第 i 个位置只能使用自身及之前的位置。对于固定模型和相同的位置编码设置，如果前缀 token 序列与位置一致，前缀各位置的隐藏状态不受后续后缀影响。沿网络各层推下去，这些位置的 K/V 也相同，因此可以复用已经保存的 KV。这里复用的是前缀的中间计算结果，不是直接复用整段回答。[1]

命中公共前缀后，仍需计算 64 token 后缀的隐藏状态和 K/V；后缀注意力也要读取已有前缀 KV。生成阶段还要逐步计算新 token 并追加 KV。分散组虽然与共享组每对输入只有首 token 不同，但完整前缀已经不同；后续 token 可能关注首 token，不能把它们当作相同上下文直接复用。本次分散组实际 `cached_tokens` 全部为 0。

4 数据能说明什么

共享组每条请求命中 2048 token，实际 Prefill 仅计 64 token。总输入为 $32 \times 2112 = 67,584$ ，命中为 $32 \times 2048 = 65,536$ ，所以命中率为 $2048 / 2112 = 96.97\%$ ；Prefill 总数为 $32 \times 64 = 2,048$ 。与分散组相比，该计数减少 96.97%，符合负载设计。

共享组吞吐量为 48.35 请求/s，分散组为 7.94 请求/s，前者约为后者的 6.09 倍。TTFT p50 从 533.409 ms 降到 43.313 ms，降低约 91.88%。减少前缀 Prefill 工作量后，首个 token 可以更早返回，这与测量方向一致。

TPOT p50 从 31.016 ms 降到 7.646 ms，但不能解释为共享前缀免除了 Decode。本次多请求并发下，较少的 Prefill 工作量可能减少与 Decode 竞争资源及调度等待，这是对结果的解释，未通过 GPU 性能剖析单独验证。TTFT 与 TPOT 处于不同阶段，变化比例不必相同。

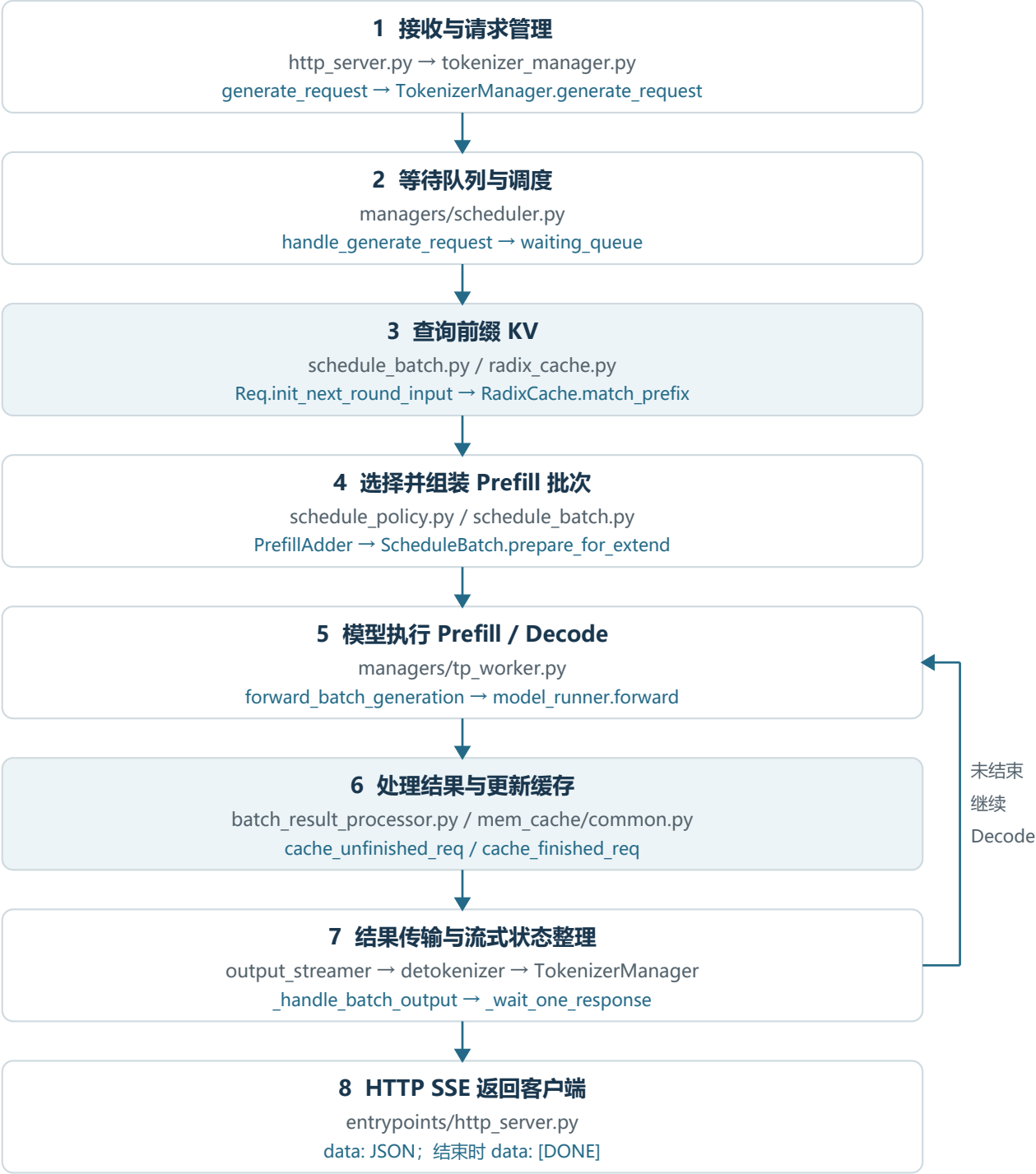
5 结论与限制

两组均全部成功，输入和输出长度一致；共享组缓存命中率更高、Prefill token 更少，任务一的完成条件满足。这次只测了 32 + 32 条人工构造输入，共享组先测，分散组后测，没有重复运行、交换组顺序或统计置信区间。客户端时间包含 HTTP、流式传输和处理开销，结果只描述本机这一次固定配置，不能直接推到其他模型或真实业务负载。

[1] Vaswani et al. Attention Is All You Need, 2017. [arXiv:1706.03762](https://arxiv.org/abs/1706.03762)。式中额外写出因果掩码，以对应本实验的自回归生成。

任务二：请求主流程图

图 1 原生 /generate 的主要处理路径（按 v0.5.14 源码简化）



调度参考主线是 event_loop_normal → get_new_batch_prefill; 服务器启用 overlap 时可走 event_loop_overlap。图中表示主流程和生成循环，不表示所有进程之间都按同一个线程同步执行。Radix Cache 插入和流式输出由结果处理环节触发。

任务二：源码说明

请求进入等待队列：HTTP /generate 把请求交给 TokenizerManager.generate_request；使用 input_ids 时不再按聊天模板构造文本。TokenizerManager 管理请求状态并发送给 Scheduler。

Scheduler.handle_generate_request 创建 Req，普通生成请求经 _add_request_to_queue 加入 waiting_queue，再由调度循环选择可以执行的批次。

前缀匹配减少本轮 Prefill：get_new_batch_prefill 的内部逻辑调用 Req.init_next_round_input，再调用 tree_cache.match_prefix，拿到 prefix_indices。PrefillAdder 根据资源预算决定能接收哪些请求；ScheduleBatch.prepare_for_extend 用输入序列去掉已匹配前缀后的部分构建新增输入。共享组命中 2048 token，因此每条剩余 Prefill 输入计 64 token；分散组命中为 0，需处理完整 2112 token。

新 KV 写回缓存：模型工作进程执行批次后，结果处理逻辑对未结束请求调用缓存维护入口，对结束请求调用 release_kv_cache。对应的 RadixCache.cache_unfinished_req 和 cache_finished_req 把 token 序列与已有 KV 索引插入树中，并处理重复或多余分配和节点引用。缓存保存的是 token 前缀到 KV 的映射，不是重新复制一份完整回答。

流式返回：结果处理环节调用 output_streamer，经解码相关进程传回 TokenizerManager；_handle_batch_output 整理文本、输出 token 和 meta_info，等待响应逻辑向 HTTP 层提供增量结果。HTTP 层把结果封装为 SSE 的 data 事件，生成结束后发送 [DONE]。实验客户端从这些事件取得 completion_tokens、cached_tokens 和到达时间。

关键源码文件 (python/sglang/srt/ 下)	核对入口
entrypoints/http_server.py	generate_request (769 行)
managers/tokenizer_manager.py	generate_request (576) ; _handle_batch_output (1839)
managers/scheduler.py	event_loop_normal (1505) ; _add_request_to_queue (2258) ; get_new_batch_prefill (2702)
managers/schedule_batch.py	Req.init_next_round_input (1123) ; prepare_for_extend (2008)
managers/schedule_policy.py	PrefillAdder (425)
managers/tp_worker.py	TpModelWorker.forward_batch_generation (482)
managers/scheduler_components/ batch_result_processor.py	结果处理：缓存维护和 stream_output 调用
mem_cache/common.py / radix_cache.py	release_kv_cache (629) ; match_prefix (358) ; cache_finished_req (438) ; cache_unfinished_req (485)

名称核对：题目主线里的 match_prefix_for_req 在此次核对的 scheduler.py 与 schedule_batch.py 中未找到同名函数；本版本实际定位为 Req.init_next_round_input → tree_cache.match_prefix。上述位置按官方 release/v0.5.14 分支的固定提交核对；详细链接在 src/target2/source-map.md。

源码来源（官方 release/v0.5.14 分支固定提交）：[HTTP 接口](#)；[请求管理](#)；[调度](#)；[批次](#)；[预算](#)；[模型执行](#)；[结果处理](#)；[缓存维护](#)；[RadixCache](#)。

作业感受

(1) 完成过程与 AI 辅助

这次实验复用了第一次挑战中搭建的 Ubuntu 和 SGLang 环境。我在 Ubuntu 终端运行模型服务和实验客户端，把终端截图及原始结果交给 AI 核对。AI 帮助整理实验要求、编写客户端、核对统计结果并生成报告。操作中，我询问过应该在哪个 Ubuntu 终端执行命令，也曾误把结果中的 `assignment_checks_passed` 输入为命令，出现 `command not found`。之后通过解释区分了执行命令和结果显示。任务二阅读时，我继续询问 CUDA、TokenizerManager、KV 前缀、`waiting_queue`、`ScheduleBatch`、并发数及 Prefill/Decode 的含义，再结合讲义与源码链接学习。

(2) 最困难的工作及解决办法

任务二里，我遇到了不少困难，最难的还是看代码。我没有系统学过 Python，只自学过一些基础，所以第一次打开 GitHub 上的代码时，很多地方都看不懂。比如 `req`、KV、Prefill 这些词，我连它们是什么意思都不知道，更不知道该从哪里开始读。

后来，我尝试让 AI 帮我解释代码。最开始的解释我也不太能看懂，就不断调整提示词，让它用适合初学者的方式讲，最后生成了一份讲义。讲义里有对应代码的链接，我可以一边看解释，一边回到代码里找，这样才慢慢看懂了一些。比如，我知道了 `obj` 在这里是收到的请求对象，里面放着输入和生成参数等信息；KV 中的 K 和 V 分别是 Key 和 Value，是模型在注意力计算中产生的数值向量。以前我只知道缓存能减少重复计算，这次才对它具体保存什么、怎么被使用有了一些理解。

看完讲义后，我又让 AI 出了四个小问题，检查自己有没有看懂。有些内容阅读时觉得理解了，但真要回答问题时，还是说不清楚。我就回去再看对应的代码和讲义，并且继续问 AI。这样反复几次后，我逐渐理清了任务二的主要流程。现在我还不能独立看懂所有代码，但至少面对这次实验里的部分 Python 代码时，知道该从哪里开始读，也能大概理解它们在做什么。

(3) 对从零到一科研工作的启发

我觉得包含对未知领域的研究，对已知领域的优化与应用

我觉得，像这次借助 AI 从头学习、尝试读懂代码的工作，并没有一开始想的那么难。现在获取知识的途径很多，遇到不懂的地方，可以让 AI 帮忙解释，一次没听懂还可以接着问。不过，学会已有的东西和自己想出新的东西还是有区别的。我觉得后者才是更难的部分。

做这类工作，基础还是要先弄清楚。比如这次读代码，我得先了解一些 Python 的基本用法，再去弄懂代码里的专业术语。不然，即使 AI 给出了解释，我也不一定能看明白。所以工具虽然方便，该学的基础知识还是得学。

以后做科研，我也会继续用 AI 帮助自己学习。至于它将来能发展到什么程度，会不会也能提出有创造性的想法，我现在说不准。对我来说，先把现有的工具用好，同时多想想自己发现了什么问题、有没有新的思路，是比较实际的做法。